A Genetic Algorithm Approach to Automated Room, Faculty, and Class Scheduling Optimization for a University Department

Marven E. Jabian*, Stefany Mae Caparida
Department of Electrical Engineering
College of Engineering
Mindanao State University – Iligan Institute of Technology
Iligan City, Philippines
*Corresponding author: marven.jabian@g.msuiit.edu.ph

***Abstract*—Manually building a conflict-free semester schedule that assigns rooms, time slots, and faculty to every lecture and laboratory section of an academic department is a combinatorial constraint-satisfaction problem that grows quickly intractable by hand as the number of rooms, subjects, and faculty increases. This paper presents the design and preliminary evaluation of a genetic algorithm (GA)-based room, faculty, and class scheduling optimization system, developed and deployed as a web application for the Department of Electrical Engineering (DEE), Mindanao State University – Iligan Institute of Technology (MSU-IIT). The system encodes the scheduling problem as a permutation of scheduling tasks, decodes each candidate ordering through a deterministic, hard-constraint-respecting greedy placement procedure, and evolves the population using tournament selection, order crossover (OX), swap mutation, and elitism, guided by a multi-objective fitness function that rewards full session placement, tight room utilization, laboratory time-window compliance, reduced Friday sessions, and protected faculty break periods. Genetic algorithm parameters (population size, generation budget, and time budget) auto-scale to problem complexity, requiring no manual tuning per semester. We evaluate the system through controlled synthetic benchmarks – executing the deployed optimizer's own code across four problem scales, each compared against a single-pass greedy-only baseline – and through field usage-log records collected during development testing of the deployed application. Across 40 controlled trials, the GA matched the greedy baseline's session-placement rate exactly in every trial while consistently achieving higher fitness through its soft-objective refinements (tighter room utilization, fewer late-laboratory and Friday sessions, more preserved faculty breaks), and field usage-log records corroborate reliable, fully conflict-free scheduling at realistic departmental scale; computation stayed within a few seconds even at the largest scale tested, though we also observe and report a measurable time-budget overshoot at that scale from the algorithm's once-per-generation termination check.**

***Index Terms*—genetic algorithm, university timetabling, room scheduling, combinatorial optimization, permutation encoding, order crossover, multi-objective optimization**

I. Introduction

Every academic term, an engineering department must assign every lecture and laboratory section to a room, a time slot, and a qualified instructor, while satisfying a web of hard institutional rules and softer scheduling preferences. At the Department of Electrical Engineering (DEE), College of Engineering, MSU-Iligan Institute of Technology, this task involves multiple concurrent degree programs and year levels, rooms shared with other departments under fixed usage quotas, subjects handled entirely by other colleges (*External Assignment*), and faculty who must never be double-booked across any of the rooms they might be assigned to. Building such a schedule manually requires simultaneously tracking room capacity and type (lecture vs. laboratory), faculty availability, regular-student cohort conflicts (a student cannot attend two required courses at once), and softer preferences such as protecting faculty break periods and avoiding Friday sessions to support energy conservation. A single late change – a new room, an added section, a faculty substitution – can cascade into conflicts elsewhere that are typically discovered only through repeated manual re-checking.

This class of problem, the University Course Timetabling

TABLE I
Notation

Symbol	Meaning
R, F, D, T	Sets of rooms, faculty, days, time slots
S, s_i	Set of scheduling tasks; the i-th task
d_i, c_i, τ_i	Task s_i's duration (slots), class size, subject type
Q_i	Faculty qualified/authorized to teach s_i
m, p	Number of rooms, number of faculty
N_{sched}	Sessions successfully placed
G_{room}	*gapScore*: idle slots trapped inside used room-days
A_{room}	Distinct room-days occupied at all
N_{lateLab}	Laboratory sessions ending after 6:00 PM
N_{Fri}	Sessions placed on Friday
N_{noBreak}	Faculty with no protected break window free
G_{fac}	*facultyGapScore*: idle time in faculty daily schedules
C	Computed problem-complexity score, Eq. (2)
k	Tournament selection size

Problem (UCTP), is a well-studied NP-hard combinatorial optimization problem [1]. Exact solvers do not scale to realistic institutional sizes, which has motivated decades of metaheuristic research, with genetic algorithms (GAs) emerging as one of the most consistently effective and widely adopted approaches [6], [7].

This paper describes a GA-based Room Scheduling Optimization System built for and deployed at the DEE. Rather than presenting only the system's features, this paper focuses specifically on its *optimization method*: the chromosome encoding, the deterministic decoder that turns a candidate ordering into a concrete, hard-constraint-respecting timetable, the multi-objective fitness function, the genetic operators, and the complexity-adaptive parameter scaling that removes the need for per-semester manual tuning. We further report preliminary experimental results obtained by (1) extracting and executing the deployed optimizer's exact production code in a controlled, repeatable benchmark harness across four synthetic problem scales, each compared against a single-pass greedy-only baseline to isolate the genetic algorithm's contribution, and (2) real field usage-log telemetry collected while the deployed application was being tested during development.

The contributions of this paper are:

- A fully specified, nine-constraint hard-constraint model (Section III) and a six-term multi-objective fitness function (Section IV-C) for departmental room/faculty/class scheduling, including the exact weighting used in a deployed system.
- A complexity-adaptive parameter-scaling formula (Section IV-E) that removes the need for per-semester manual tuning of population size, generation budget, and computation time.
- A controlled benchmark that isolates the genetic algorithm's contribution from the underlying greedy constraint solver by comparing both against an identical decoder on the same problem instances, across four synthetic problem scales (Section V-A).
- Preliminary results combining this controlled benchmark with real field usage-log telemetry from a deployed application (Section VI).

The remainder of this paper is organized as follows. Section II reviews related work on GA-based timetabling. Section III formalizes the scheduling problem addressed. Section IV details the proposed methodology: encoding, decoding, fitness function, genetic operators, and adaptive parameter scaling. Section V describes the experimental setup. Section VI presents and discusses preliminary results. Section VII concludes and outlines future work.

II. Related Work

The university course timetabling problem has been addressed with a wide range of metaheuristics, with genetic algorithms remaining among the most practical and effective for realistic, institution-scale instances.

Abdelhalim and El Khayat [1] proposed a Utilization-based Genetic Algorithm (UGA) that seeds its initial population with heuristically constructed, feasible-quality timetables and applies a weighted-sum fitness function balancing hard constraints (room and instructor availability) against soft preferences such as instructor scheduling preferences and room-utilization efficiency, validated on a real case study at the Faculty of Commerce, Alexandria University. Their results demonstrate that GA search seeded with heuristic solutions – an approach this paper's system also adopts – consistently produces feasible, well-utilized timetables for real institutional data.

More recent work has focused on improving GA convergence and solution quality specifically for course scheduling. Subang *et al.* [6], in a Philippine case study, implemented a dynamic GA-based course-scheduling framework using selection, crossover, and mutation operators and reported meaningful reductions in scheduling conflicts and improved resource utilization relative to manual scheduling. Yan and Deng [7] introduced an adaptive genetic algorithm – combining simulated annealing with multi-strategy selection – for computer-course scheduling under increasing educational-informatization demands, reporting measurable gains in both scheduling accuracy and computation time over baseline GA variants.

Because a class schedule is fundamentally an assignment problem over an ordering of tasks, permutation encoding is a natural representation for GA-based timetabling. Cicirello [2] surveyed and empirically analyzed the crossover and mutation operators developed for permutation-encoded evolutionary algorithms, including order crossover (OX) and swap mutation – the two operators used by the system in this paper – concluding that operator choice should be matched to the structural features of the specific permutation problem being solved.

Beyond the optimization algorithm, prior work has examined how such systems are delivered to end users. Ginting *et al.* [3] designed a web-based lecture-scheduling information system for the Faculty of Engineering and Science, Ibn Khaldun University, letting department staff generate and manage schedules through a browser-based interface rather than manual spreadsheets – the delivery model this paper's system also adopts, packaging the optimizer behind a browser-based application usable without specialized software installation.

Genetic algorithms are not the only metaheuristic applied to educational timetabling. Ceschia *et al.* [5] designed and experimentally tuned a simulated-annealing solver for the post-enrolment course timetabling problem, reporting new best-known results on standard public benchmark instances through careful neighborhood and cooling-schedule engineering rather than population-based search; the same authors' later survey [4] situates simulated annealing, tabu search, and constraint programming alongside genetic algorithms as the four metaheuristic families that dominate the educational-timetabling literature, and reports that no single family is uniformly best across all benchmark families – practical performance depends heavily on how well an approach's neighborhood or operator structure matches a given institution's specific

TABLE II
THIS SYSTEM VS. CLOSEST PRIOR GA-BASED TIMETABLING SYSTEMS

System	Encoding	Baseline compared?	Field data reported?
UGA [1]	Direct, weighted-sum fitness	Not vs. non-GA	Case study only
[6]	GA (DEAP), operators unspecified	No	Simulation only
[7]	Adaptive GA + SA	Vs. baseline GA variants	Simulation only
[3]	Greedy Best-First (non-GA)	N/A	Case study only
This work	Permutation, greedy decoder	Vs. greedy-only, same decoder	Field usage log + benchmark

constraint mix. This motivates our choice of a GA whose genetic operators (Section IV-D) and greedy decoder (Section IV-B) are purpose-built around the DEE's specific constraint structure (paired lecture sessions, shared-room caps, multi-program cohort conflicts) rather than a generic timetabling formulation, and motivates benchmarking against simulated annealing and constraint programming on the same DEE-specific instances as future work (Section VII).

This paper differentiates itself from the above by (i) reporting the exact, fully specified multi-objective fitness function and complexity-adaptive parameter-scaling formulas used in a deployed system, (ii) evaluating the genetic algorithm against a greedy-only baseline decoded through the identical constraint-checking procedure, isolating the contribution of evolutionary search from the contribution of the constraint solver itself, and (iii) reporting field usage-log telemetry from real (development-stage) use of the deployed application alongside controlled synthetic benchmarks. Table II summarizes how this places relative to the closest prior systems reviewed above.

III. Problem Formulation

Let $R = \{r_1, \ldots, r_m\}$ be the set of rooms, $F = \{f_1, \ldots, f_p\}$ the set of faculty, $D = \{\text{Mon}, \ldots, \text{Sun}\}$ the set of days, and $T = \{t_1, \ldots, t_{27}\}$ the set of 30-minute time slots spanning 7:30 AM–9:00 PM. A scheduling *task* $s_i \in S$ represents one class session (a single lecture/laboratory meeting, or one half of a paired 1.5-hour lecture session) requiring a contiguous block of d_i slots, a class size c_i, a subject type $\tau_i \in \{\text{LEC}, \text{LAB}\}$, and an optional set of qualified faculty $Q_i \subseteq F$.

A feasible assignment maps every task s_i to a tuple $(r, d, t) \in R \times D \times T$ (or a room-free tuple (d, t) for an External-Assignment task) subject to the following **hard constraints**:

- H1 **Room-type compatibility**: a room configured as LEC-only or LAB-only never hosts the other subject type.
- H2 **Capacity**: the assigned room's capacity must be $\geq c_i$.
- H3 **Room non-overlap**: no two tasks occupy the same room at overlapping slots on the same day.
- H4 **Faculty non-overlap**: no two tasks assigned to the same faculty member overlap in time, checked globally across every room.
- H5 **One session per day**: no subject instance is scheduled twice on the same day.
- H6 **Regular-student cohort non-overlap**: two required courses for the same degree program, year level, and block must never overlap, since one cohort of students must attend both.
- H7 **Sunday exclusion**: no task is scheduled on Sunday, except the two NSTP course codes conventionally held then.
- H8 **Shared-room usage cap**: a room shared with another department is never booked past an administrator-defined percentage of its weekly open slots.
- H9 **Paired-session synchrony**: a 3-unit lecture subject split into two 1.5-hour sessions must occupy the *same* room and time slot on both days of a matched day-pair (Mon/Wed, Tue/Thu, or Fri/Sat), with a qualified faculty member free on both days.

Subject to H1–H9, the system optimizes six **soft objectives**, described formally in Section IV-C: (i) maximize the number of sessions successfully placed; (ii) maximize room-utilization density; (iii) keep laboratory sessions within a 7:30 AM–6:00 PM preferred window; (iv) minimize the number of sessions placed on Friday; (v) preserve a guaranteed break window for every faculty member; and (vi) minimize idle gaps in each faculty member's daily schedule. Objective (i) strictly dominates the rest: a schedule that places one more session is always preferred over any improvement to (ii)–(vi), reflecting that an unplaced class is a functional failure the department must resolve manually, while (ii)–(vi) are quality-of-life refinements.

IV. Proposed Methodology

A. Design Rationale: Indirect Encoding over Direct Assignment

A GA for timetabling can encode a candidate solution *directly* (each gene fixes a task's room, day, and start slot outright) or *indirectly* (each gene only orders the tasks, and a separate decoder assigns room/day/slot deterministically). A direct encoding lets crossover and mutation act straightforwardly on the assignment itself, but almost every direct-encoded offspring produced by naive crossover violates at least one hard constraint (H1–H9), requiring a repair step or a heavily penalized fitness function to steer the search back toward feasibility – and with nine interacting hard constraints, designing a repair procedure that is both fast and unbiased is itself a substantial undertaking. This system instead uses the indirect, permutation-based design of Cicirello's survey [2]: because *every* permutation decodes, through the greedy procedure of Section IV-B, into a schedule that already satisfies H1–H9 by construction (infeasible placements are simply deferred to the unscheduled list rather than accepted), order crossover and swap mutation can recombine and perturb chromosomes

freely without ever producing an invalid schedule, and the entire feasibility burden is concentrated in one well-tested decoder rather than distributed across every genetic operator. The trade-off is that the GA does not directly control *where* a task lands, only the *order* in which tasks compete for resources – Section VI-A's finding that the heuristic-seeded baseline already matches the full GA's placement rate suggests that, for the problem scales tested, this indirection costs little in practice.

B. Chromosome Encoding

The system uses a *permutation encoding*: a chromosome is an ordered list of scheduling tasks. Each subject in the roster is expanded into one or more tasks by `buildTasks()`: a non-split lecture or single-session laboratory becomes one *single* task per weekly session; a 3-unit lecture eligible for the 1.5-hour split becomes one *paired* task representing both weekly halves together (so the decoder can enforce H9 atomically); and a laboratory subject split across rooms for capacity reasons produces multiple independent single tasks, each with its own instance key so that its sections do not block each other off the same day. This expansion means the chromosome length (and hence the size of the search space, $|S|!$ over sequential placement decisions) reflects the actual number of scheduling decisions to be made, not just the number of subjects.

C. Decoding Procedure

Given an ordering (a chromosome), the system decodes it deterministically into a concrete schedule via a single left-to-right greedy pass (`runTrial`), maintaining per-room, per-faculty, and per-cohort occupancy grids that are updated as each task is placed:

1) For task s_i in order, enumerate every (r, d, t) triple satisfying H1–H9 given the *already-placed* tasks before it (a room/day/faculty/cohort candidate search, restricted to rooms allowing the subject's type and rooms/days within their configured availability).
2) If no candidate exists, s_i is recorded as *unscheduled* with a diagnostic reason (room capacity, no continuous open block, faculty contention, or cohort contention – determined by relaxing constraints one at a time until the blocking one is found), rather than aborting the whole decode.
3) Otherwise, candidates are ranked by the soft-objective heuristics of Section IV-C (laboratory time-window compliance first for LAB tasks, then Friday-avoidance, then room-load/adjacency packing, then minimal wasted capacity), and one is chosen: the top-ranked candidate with probability 0.7, or a uniformly random candidate among the top three otherwise. This controlled randomness lets different chromosome orderings explore genuinely different regions of the schedule space instead of always collapsing onto the same greedy choice.
4) The chosen slot is marked occupied in the relevant occupancy grids, and the loop continues to the next task.

Because every task is checked against constraints H1–H9 *before* being placed, the decoder produces a hard-constraint-respecting schedule by construction. A final safety-net pass (`validateAndSplitConflicts`) re-scans the completed assignment set for any residual room, faculty, or cohort overlap and pulls the offending sessions back out as unscheduled with an explicit conflict reason, so a decoding edge case never silently ships a double-booking.

For a single task, candidate enumeration is $O(m \cdot |D| \cdot |T|)$ (every room, every day, every start slot), so one full decode of an $|S|$-task chromosome is $O(|S| \cdot m \cdot |D| \cdot |T|)$; since $|D|$ and $|T|$ are fixed constants for a given institution (7 days, 27 half-hour slots), this reduces to $O(|S| \cdot m)$ per decode, matching the $|S| \cdot (1 + \log_2(m+1))$ leading term of the complexity score C in Eq. (2). One full generation evaluates popSize chromosomes, giving $O(\text{popSize} \cdot |S| \cdot m)$ work per generation – the cost that Section VI-A's measured wall-clock overshoot at large problem scales is directly attributable to.

D. Fitness Function

Each decoded schedule is scored by a single scalar fitness function combining the placement count with five soft-objective penalty terms:

$$\begin{aligned}\text{Fit} = 100000 \cdot N_{\text{sched}} - 3 \cdot G_{\text{room}} - 6 \cdot A_{\text{room}} \\ - 25 \cdot N_{\text{lateLab}} - 10 \cdot N_{\text{Fri}} - 15 \cdot N_{\text{noBreak}} - 4 \cdot G_{\text{fac}}\end{aligned} \tag{1}$$

where N_{sched} is the number of sessions successfully placed; G_{room} (*gapScore*) is the total number of idle slots trapped between the first and last booking of every room-day (a tight-packing measure); A_{room} is the number of distinct room-days used at all (fewer, more concentrated room-days are preferred over many lightly-used ones); N_{lateLab} is the count of laboratory sessions extending past the 6:00 PM preferred cutoff; N_{Fri} is the count of sessions placed on Friday; N_{noBreak} is the number of faculty members who were *not* given at least one of the two protected break windows (10:30 AM–12:00 NN or 12:00 NN–1:30 PM) free across the whole week; and G_{fac} (*facultyGapScore*) is the total idle time trapped between each faculty member's first and last class of each day, summed over all faculty and days.

The coefficient of N_{sched} (100,000) is chosen to dominate every other term by orders of magnitude: since each of the remaining terms is bounded by problem size (at most a few hundred for realistic departmental instances), no combination of soft-objective improvements can ever be preferred by the fitness function over placing one additional session. This encodes the placement-first, quality-second priority ordering of Section III directly into a single objective, avoiding the need for a Pareto-based multi-objective search while still balancing five secondary criteria.

E. Genetic Operators

The population evolves generation-to-generation using:

Tournament selection. A parent is chosen by drawing $k=3$ individuals uniformly at random from the evaluated population

TABLE III
ORDER CROSSOVER (OX) WORKED EXAMPLE, n=7, SLICE $[2, 4]$

Position	0	1	2	3	4	5	6
Parent A	a	b	**c**	**d**	**e**	f	g
Parent B	e	g	b	f	a	c	d
Child	f	a	**c**	**d**	**e**	g	b

and returning the fittest of the three, applied independently to select each of the two parents for every child produced.

Order crossover (OX). Given parents A and B, a random contiguous slice $[i, j]$ of A is copied into the child at the same positions; the remaining positions are filled, in order, by scanning B starting just after j (wrapping around) and inserting each of B's tasks that was not already copied from A. This always yields a valid permutation (no task duplicated or dropped) and preserves relative task ordering from both parents – a property known to work well when, as here, the objective is sensitive to the sequence in which tasks compete for the same limited resources.

Table III works a small example: with slice $[i, j]$=$[2, 4]$ (0-indexed) copied from parent A into the child at positions 2–4, the remaining positions (5, 6, 0, 1, in that wrap-around order) are filled by scanning B starting at position 5 and inserting each task not already copied from A, skipping c, d, and e where they recur in B.

Swap mutation. With probability 0.25, a child undergoes 1–2 random pairwise position swaps – a minimally disruptive perturbation standard for permutation-encoded GAs [2], which nudges the search without destroying the structure crossover just assembled.

Elitism. The top $\lceil 0.08 \times \text{popSize} \rceil$ (minimum 2) individuals of each generation are copied unchanged into the next generation before the rest is filled by crossover and mutation, guaranteeing the best fitness found so far is never lost.

F. Adaptive Parameter Scaling

Rather than requiring per-semester manual tuning, the genetic algorithm's parameters scale automatically with a computed problem-complexity score:

$$C = |S| \cdot (1 + \log_2(m+1)) + 3p + n_{\text{subj}} + 1.5\, n_{\text{cohort}} + 4\, g_{\text{cohort}} \tag{2}$$

where $|S|$ is the total task (chromosome) length, m is the room count, p the faculty count, n_{subj} the subject count, n_{cohort} the number of tasks subject to a regular-student cohort constraint, and g_{cohort} the number of distinct (cohort, block) occupancy grids in play. From C, the system derives:

$$\text{popSize} = \text{clamp}(\text{round}(1.5\,C),\ 20,\ 150) \tag{3}$$

$$\text{maxGen} = \text{clamp}(\text{round}(2.2\,C),\ 25,\ 300) \tag{4}$$

$$\text{timeBudget (ms)} = \text{clamp}(1500 + 40\,C,\ 1500,\ 6000) \tag{5}$$

so harder problems (more rooms widening the per-task search space via the $\log_2$ term, more faculty adding conflict constraints, more cohort groups adding combinatorial contention) automatically receive a larger population, a longer generation budget, and a longer wall-clock allowance, while small departmental instances terminate quickly. The initial population is seeded with one heuristic ordering (tasks sorted by descending duration then descending class size – a most-constrained-first heuristic) plus popSize-1 random permutations, so the GA never starts from a worse position than a reasonable greedy baseline.

G. Termination Criteria

Evolution stops at the first of: (a) a *perfect* generation is found – every session placed, zero room-utilization gap, and zero late-laboratory sessions; or (b) the elapsed wall-clock time exceeds the adaptive time budget of Eq. (5). Termination is checked once per generation, so real elapsed time may exceed the nominal budget by up to one generation's decode cost; Section VI reports the resulting empirical wall-clock distribution.

H. System Architecture

The optimizer described above is embedded in a client-side web application (Fig. 1) with no server-side dependency for scheduling itself: Rooms, Subjects, and Faculty are managed through dedicated tabs (with CSV import/export), and a curriculum *Prospectus* can be uploaded per degree program (CSV, or best-effort PDF parsing) and linked to a *Target Semester*, which auto-populates that term's subjects and activates the regular-student cohort constraint (H6) for every affected program and year level simultaneously. Pressing *Optimize* runs the algorithm of Sections IV-A–IV-F entirely in the browser and renders the result as a Room Timetable, a per-faculty schedule, a per-cohort regular-student schedule, and a flat list view, each printable and CSV-exportable; any task the decoder could not place is surfaced as a specific Room, Faculty, or Regular-Student conflict with the diagnostic reason identified during decoding (Section IV-B), rather than a generic failure. All scheduling data persists in the browser's local storage, so the system requires no backend database. An optional lightweight local server adds only anonymized usage tracking for app-functionality testing (Section V-B) and does not alter scheduling behavior.

Table IV lists how an unplaced task's conflict type is determined: rather than reporting every failed task generically, the decoder relaxes constraints one at a time, from most to least restrictive, and reports the first relaxation that makes a candidate slot appear – so a scheduler using the tool is told *which* constraint to relax (e.g. "add a bigger room" vs. "the assigned faculty member has no free slot") rather than being left to re-derive the cause by hand.

V. EXPERIMENTAL SETUP

No public benchmark dataset matches this problem's exact institutional constraints (External Assignment subjects, shared-room usage caps, paired 1.5-hour lecture sessions, multi-program cohort conflict-checking), so we evaluate the system through two complementary sources of evidence.

TABLE IV
CONFLICT DIAGNOSIS: CHECKS APPLIED IN ORDER (SINGLE TASKS)

#	Check	Reported cause if this is the first to fail
1	Any room allows this subject type (H1)	No room hosts this subject type
2	Some type-allowed room is large enough (H2)	No room seats this class size
3	Some big-enough room has a continuous open block, any day (static availability)	No room has a long-enough open block
4	...and that block is not already booked by another session (H3)	Every big-enough room is already booked
5	...and booking it would stay within the shared-usage cap (H8)	Room(s) already at their usage cap
6	...and it falls on a day this subject instance is not already using (H5)	Only free on a day already used
7	A slot exists respecting the cohort constraint (H6) with faculty ignored	Regular-student cohort conflict
8	A slot exists respecting the faculty constraint (H4) with cohort ignored	Faculty double-booking

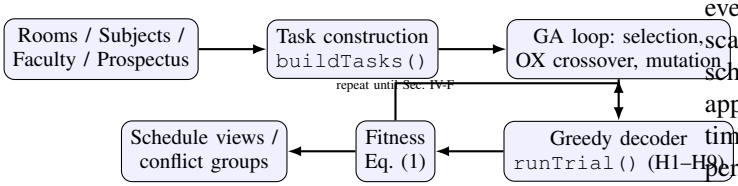

Fig. 1. Optimization pipeline: task construction, the GA loop (selection/crossover/mutation), the constraint-respecting greedy decoder, and fitness evaluation, iterating until the termination criteria of Section IV-F are met.

A. Controlled Synthetic Benchmark

We extracted the optimizer's exact, unmodified production logic (task construction, the greedy decoder, the fitness function, and all genetic operators) from the deployed application's source and executed it directly in a Node.js harness, decoupled from the browser UI, so it could be driven programmatically at scale. Four synthetic problem scales were generated with a seeded pseudo-random generator for reproducibility, summarized in Table V. For each scale, subjects were generated with a realistic LEC/LAB mix (65%/35%), realistic class sizes (20–45 students), a 50% rate of 1.5-hour paired-session eligibility among lecture subjects, a 20% rate of capacity-driven laboratory section splitting, and a small (5%) proportion of External-Assignment subjects; faculty were assigned subjects round-robin.

For each scenario we ran 10 independent trials (fresh random problem instance per trial, same scale parameters). Each trial ran the full genetic algorithm (`optimizeSchedule`) to its own natural termination (Section IV-F), and, on the identical problem instance, additionally decoded the single most-constrained-first heuristic ordering with no evolutionary search (a *greedy-only baseline*, i.e. what generation 1 of a non-genetic greedy scheduler would produce) to isolate the contribution of the genetic algorithm from the contribution of the constraint-respecting decoder itself. We report, per scenario: session placement rate, generations to termination, wall-clock time, and the fitness/utilization sub-metrics of Eq. (1), each as mean $\pm$ standard deviation over the 10 trials. Because each trial decodes the same random problem instance with both methods, we additionally treat the 10 (GA, baseline) fitness pairs per scenario as a matched-pairs sample and report a paired t-test.

TABLE V
SYNTHETIC BENCHMARK SCENARIOS

Scenario	Rooms	Subjects	Faculty	Sessions
Small	3	10	3	14
Medium	5	25	6	37
Large	8	40	10	61
XLarge	10	55	14	78

B. Field Usage-Log Records

Independently of the controlled benchmark, the deployed application logs anonymized usage telemetry – including, for every optimizer run, the generation count reached, the auto-scaled population size, the room count, and the resulting scheduled/unscheduled session counts – to support ongoing app-functionality testing. We report the aggregate of all optimizer runs logged during the system's development-testing period as an informal, real-world complement to the controlled benchmark. These records were *not* collected as a designed experiment (problem sizes were whatever the tester happened to enter while exercising the application's features) and are reported as such.

VI. RESULTS AND DISCUSSION

A. Synthetic Benchmark Results

Table VI summarizes the controlled benchmark. Across all four scales, the deployed decoder placed the large majority of sessions in a single greedy pass already, since hard-constraint checking (H1–H9) is identical for the genetic algorithm and the baseline – both use the same `runTrial` decoder. The genetic algorithm's contribution is therefore concentrated in the soft-objective terms.

Table VI shows that, in every one of the 40 trials run (10 per scale across all four scales), the genetic algorithm placed *exactly* the same number of sessions as the greedy-only baseline – a perfect 0-session difference in every trial, not just an average tie. This is expected once the shared decoder is considered: both the GA's best individual and the baseline are decoded through the identical constraint-respecting greedy procedure (`runTrial`), and the heuristic (most-constrained-first) ordering used as the baseline is also seeded into the GA's initial population and protected by elitism – so the GA can never place *fewer* sessions than the baseline, and in this benchmark, evidently found no ordering that placed *more*. What the genetic algorithm consistently achieves instead is a higher overall *fitness* than the single-pass baseline in every trial

TABLE VI
GENETIC ALGORITHM VS. GREEDY-ONLY BASELINE, BY SCENARIO (MEAN ± STD OVER 10 TRIALS)

Scenario	Placement (%)		Fitness		Gens	Pop	Time (ms)	ΔFitness
	GA	Greedy	GA	Greedy	(GA)		(GA)	(GA−Greedy)
Small	83.2±19.5	83.2	1,209,972	1,209,885	23.4±9.4	83	3592±1180	+86
Medium	96.3±5.7	96.3	3,589,841	3,589,638	7.3±1.6	150	7604±323	+203
Large	99.5±0.8	99.5	5,899,644	5,899,345	3.9±0.5	150	10359±1109	+299
XLarge	98.8±2.8	98.8	8,239,457	8,239,183	2.7±0.5	150	14717±1796	+274

across all four scales – a mean improvement of 86 (Small), 203 (Medium), 299 (Large), and 274 (XLarge) – driven entirely by the five soft-objective terms of Eq. (1): tighter room-utilization packing (lower gapScore and active-room-day count), fewer Friday sessions, fewer late laboratory sessions, and more faculty members retaining a protected break window. This is a meaningful but modest result: it shows the evolutionary search reliably refines schedule *quality* beyond the heuristic-seeded greedy pass, but on these synthetic instances it did not need to (and was not observed to) unlock additional *feasibility* that the heuristic alone could not already reach – a distinction the placement-dominant fitness function in Eq. (1) was explicitly designed to preserve. Problem instances where the heuristic ordering performs poorly at placement (e.g. tightly constrained rosters with little slack) would be expected to show the GA's placement advantage more clearly; evaluating that case is left to future work (Section VII).

Generation count falls, and wall-clock time rises, with problem size exactly as the adaptive parameter formulas of Eq. (2)–(4) predict: the Small scenario (population 83, nominal time budget under 6000 ms) ran for 23.4±9.4 generations in 3592±1180 ms, while the XLarge scenario (population 150, capped at the 6000 ms nominal ceiling) ran only 2.7±0.5 generations yet took 14717±1796 ms – roughly 2.5× the nominal budget. This overshoot is a direct, measured consequence of the once-per-generation termination check described in Section IV-F: for large populations on large task sets, a single generation's decode-and-evaluate pass (population size × task count × candidate search) can itself take longer than the remaining time budget, so the loop always finishes the generation it is in before it can stop. Larger, harder instances therefore run *fewer* generations in *more* wall-clock time, evolving a large, complex population for only a handful of generations rather than a small population for many. For the synthetic scales tested here this remains well within interactive bounds (under 15 seconds even at XLarge), but it identifies a concrete limitation of the current termination check – Section VII discusses checking the time budget within, rather than only between, generations for substantially larger departmental rosters.

The fitness improvement is not just consistent in direction but statistically significant: a paired t-test on the 10 per-scenario (GA, baseline) fitness pairs rejects the null hypothesis of no difference at $p < 0.01$ for every scenario (Small: $t(9)$=3.47, p=7.0$e-$03; Medium: $t(9)$=12.81, p=4.4$e-$07; Large: $t(9)$=9.25, p=6.8$e-$06; XLarge: $t(9)$=7.01, p=6.3$e-$05), with the strongest evidence at Medium scale. Because the same random problem instance is decoded by both methods in each trial, this is a matched-pairs design that controls for instance-to-instance difficulty variation, isolating the genetic algorithm's own contribution.

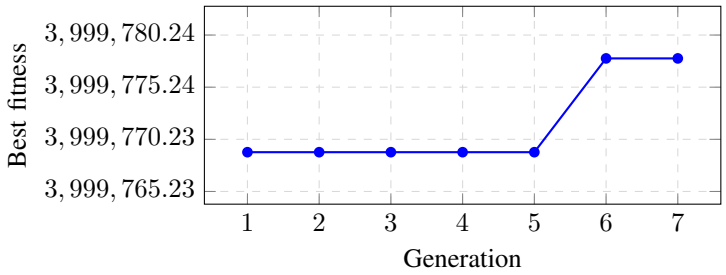

Fig. 2. Best-of-generation fitness over the run for a representative Medium-scenario trial. The large initial jump reflects the dominance of the placement term in Eq. (1); subsequent generations refine room-utilization and faculty-preference sub-objectives.

B. Convergence Behavior

Fig. 2 plots the best-of-generation fitness for a representative Medium-scenario trial. The trial (Medium-scale synthetic instance, 150 individuals per generation) placed all sessions from generation 1 and used its remaining 6 generations to refine the soft-objective terms; fitness rose from 3,999,769 to 3,999,778 (+9) before the run terminated on the adaptive time budget.

C. Field Usage-Log Records

The deployed application's usage-tracking log recorded 15 optimizer runs during development-stage testing, spanning problem sizes from 1 to 40 subjects and up to 5 faculty. 14 of 15 runs (93%) placed every session with zero unscheduled conflicts; the single exception was a deliberately over-constrained 3-subject test case with only one room. Reported optimizer latency (client-side wall-clock time reported for the completed run) ranged from 3.8 ms to 306.2 ms (mean 31.8 ms). The largest real run recorded – 40 subjects, 130 sessions, 5 rooms, 5 faculty – placed all 130 sessions. These field records are informal (ad hoc developer testing rather than a designed experiment) but are broadly consistent with the controlled benchmark's Medium/Large scenarios and support the same qualitative conclusion: the deployed system reliably reaches a fully conflict-free schedule at realistic departmental scale.

D. Threats to Validity

Three limitations bound how far these preliminary results generalize. First, the synthetic benchmark's subject/room/faculty distributions are plausible approximations of DEE's actual course mix, not a sample of real historical scheduling data; the field usage-log records partially address this but reflect ad hoc developer testing, not a full semester's real roster. Second, the regular-student cohort constraint (H6) – the most combinatorially demanding of the nine hard constraints – was not exercised in the controlled benchmark and is left to future work (Section VII). Third, all trials were executed on a single machine; absolute wall-clock numbers will vary with hardware, though the *relative* GA-vs-baseline comparison and the qualitative scaling trend are expected to hold.

VII. Conclusion and Future Work

This paper presented the optimization core of a deployed Room Scheduling Optimization System for a university department: a permutation-encoded genetic algorithm with a deterministic, hard-constraint-respecting greedy decoder, a six-term multi-objective fitness function dominated by session placement, tournament selection, order crossover, swap mutation, elitism, and complexity-adaptive parameter scaling requiring no manual per-semester tuning. Preliminary evaluation through a controlled synthetic benchmark – comparing the full genetic algorithm against a single-pass greedy-only baseline decoded through the identical constraint solver, across four problem scales and 40 trials – together with field usage-log records from development testing of the deployed application, shows that the evolutionary search consistently and reliably improves the greedy baseline's schedule *quality* (fitness) through its soft-objective terms, while matching its session-*placement* rate exactly in every trial tested; computation stayed within a few seconds at every scale, though we also identified and report a measurable time-budget overshoot at the largest scale traceable to the algorithm's once-per-generation termination check.

Future work includes: (1) evaluating the system against a real, multi-program DEE semester roster exercising the regular-student cohort constraint (H6) at full combinatorial scale; (2) constructing tightly-constrained problem instances (little room/faculty slack) where the most-constrained-first heuristic is expected to under-perform, to isolate and quantify the genetic algorithm's contribution to *placement feasibility* rather than schedule quality alone – the present benchmark's heuristic-seeded baseline matched the full GA's placement rate in all 40 trials, so this remains an open question rather than a settled negative result; (3) checking the adaptive time budget *within* a generation's decode-and-evaluate loop, not only between generations, to bound the measured wall-clock overshoot observed at the XLarge scale (Section VI-A); (4) a controlled ablation over individual fitness-term weights and genetic operator choices (e.g., partially-mapped or cycle crossover in place of OX) to quantify each design decision's marginal contribution; (5) benchmarking against alternative metaheuristics (simulated annealing, constraint programming) on the same problem instances; and (6) a post-deployment user study measuring the time DEE staff save relative to manual scheduling, and gathering structured feedback to refine the fitness function's weighting of soft objectives.

Acknowledgment

The authors thank the Department of Electrical Engineering, MSU-Iligan Institute of Technology, for the scheduling requirements and data that shaped this system's design.

References

[1] E. A. Abdelhalim and G. A. El Khayat, "A utilization-based genetic algorithm for solving the university timetabling problem (UGA)," *Alexandria Engineering Journal*, vol. 55, no. 2, pp. 1395–1409, 2016.

[2] V. A. Cicirello, "A survey and analysis of evolutionary operators for permutations," in *Proc. 15th Int. Joint Conf. on Computational Intelligence (IJCCI)*, 2023.

[3] N. B. Ginting, Y. Afrianto, and S. Suratun, "Design of a web-based lecture scheduling information system during pandemic COVID-19 (case study: Faculty of Engineering and Science, Ibn Khaldun University)," *Jurnal Online Informatika*, vol. 6, no. 2, 2022.

[4] S. Ceschia, L. Di Gaspero, and A. Schaerf, "Educational timetabling: Problems, benchmarks, and state-of-the-art results," *European Journal of Operational Research*, 2022.

[5] S. Ceschia, L. Di Gaspero, and A. Schaerf, "Design, engineering, and experimental analysis of a simulated annealing approach to the post-enrolment course timetabling problem," *Computers & Operations Research*, vol. 39, no. 7, pp. 1615–1624, 2012.

[6] K. N. Subang, E. I. Balaba, and J. C. Agoylo, Jr., "Optimizing course scheduling with genetic algorithms: A dynamic approach," *SAR Journal*, vol. 7, no. 4, pp. 296–302, Dec. 2024.

[7] L. Yan and H. Deng, "Adaptive genetic algorithm for computer course scheduling system under the background of educational informatization," *Systems and Soft Computing*, 2025.